Learning a Data-Driven Beam Search Heuristic for the Variable Gapped Longest Common Subsequence Problem

1 Overview

This work proposes a learning framework for constructing a data-driven heuristic to guide Beam Search in combinatorial optimization problems, and specifically one sequence problem that arises in bioinformatics, the *Variable Gapped Longest Common Subsequence Problem* (VGLCSP). A feed-forward neural network (FFNN) is used to parameterize the heuristic function. Instead of relying on gradient-based training, the network weights are optimized using a Genetic Algorithm (GA), precisely its popular versions called the *Random key generic algorithm* (RKGA) and a biased RKGA (BRKGA) approach.

Each individual in the GA population represents a complete set of neural network weights. The quality of an individual is evaluated by embedding the neural network inside the *Iterative Multi-source Beam Search* (IMSBS) framework (proposed in the paper for EUROCAST 2026) to guide the search in the process of forward passes of the BS, and measuring the average solution quality obtained on a set of training instances. Validation instances are used exclusively to monitor generalization.

Formally, the learning problem is defined as:

$$\max_{\theta} \frac{1}{|T|} \sum_{I \in T} f(\text{IMS_BeamSearch}(I, h_\theta)),$$

where T denotes the training set, h_θ is the neural heuristic parameterized by weights θ, and $f(\cdot)$ measures solution quality.

2 Neural Network Model

The heuristic is represented by a *multilayer perceptron* with an arbitrary number of hidden layers and user-defined units per layer. Supported activation functions include `hyperbolic tangent`, `ReLU`, and `sigmoid`.

For each layer l, forward propagation is defined as:

$$\mathbf{z}^{(l+1)} = W^{(l)}\mathbf{a}^{(l)} + b^{(l)}, \quad \mathbf{a}^{(l+1)} = \phi(\mathbf{z}^{(l+1)}),$$

where $W^{(l)}$ and $b^{(l)}$ are the weight matrix and bias vector, respectively, and ϕ denotes the activation function.

All weights and biases are flattened into a single vector, which forms the chromosome of a GA individual.

2.1 Fitness Evaluation

Given a candidate weight vector:

1. The weights are loaded into the FFNN.

2. IMSBS is executed on each training instance using the neural heuristic (guiding the search in the forward iterations as more sensible than the backward iterations).

3. The size of the best sequence obtained for each instance is recorded.

The fitness value is computed as the average solution quality over all training instances. Validation performance is computed analogously on a separate dataset and is used only for reporting.

(Q0) Speed up – evaluate fitness value by using parallelization.

2.2 Feature Extraction for the Variable-Gap LCS

To guide the search process, each partial solution (node) is represented by a fixed-length numerical feature vector capturing both the current state of the construction and the structure of the remaining search space. These features are designed to be instance-independent and comparable across problem sizes.

Let $S = \{s_1, \ldots, s_k\}$ denote the set of input sequences, and let p_i be the current matched position in sequence s_i at a given node. The vector $\mathbf{p}^L = (p_1, \ldots, p_k)$ is first normalized by the corresponding sequence lengths,

$$p_i \leftarrow \frac{p_i}{|s_i|},$$

so that progress in each sequence lies in $[0, 1]$ and becomes independent of absolute input sizes.

From this normalized position vector, four statistical descriptors are extracted: maximum, minimum, mean, and standard deviation (4 features). These quantities characterize, respectively, the most advanced and least advanced sequences, the average depth of the search, and the imbalance of progress among sequences. In addition, *the current length of the partially constructed subsequence* is included as a feature (as the fifth feature). Together, these values provide a compact description of the current search state.

To capture the feasibility of future extensions, gap-related features are computed from the positions of remaining sequences (subproblem) of input sequences. For each sequence s_i, all gap bounds associated with positions strictly after the current node position are collected. Symbolically, for each v, Gap_i^{min} is calculated as $\min\{G_i(j) \mid j = p_i^L, i = 1, \ldots, m\}$, Gap_i^{max} is calculated as $\max\{G_i(j) \mid j = p_i^L, i = 1, \ldots, m\}$, Gap_i^{avg} as $\mathrm{AVG}\{G_i(j) \mid j = p_i^L, i = 1, \ldots, m\}$, and Gap_i^{std} as $\mathrm{STD}\{G_i(j) \mid j = p_i^L, i = 1, \ldots, m\}$, where $G_1, \ldots, G_m$ are gap constraints associated to $s_1, \ldots, s_m$ respectively speed up the calc. of G-stats – preprocessing... . This gives us additionally 4 features; in overall 9 features are used for the **feature configuration 1** (F1). These statistics for gap constrains summarize the remaining flexibility of the instance: large average or maximum gaps indicate greater freedom for future matches, whereas small minimum gaps and large variability typically signal tighter constraints and increased difficulty. Additional feature $|\Sigma|$, this 10 features, created the **feature configuration 2** (F2); When adding m (number of input sequences) to F2, we obtain **feature configuration 3** (F3). (Q1) uestion: s there a need for normalization of gap values of respective subproblem... (Q2) other features?

The alphabet size $|\Sigma|$ and the number of sequences (m) as the attributes provide global structural information about the instance and help the learning model adapt to varying problem scales.

Finally, all features are standardized to zero mean and unit variance prior to being passed to the learning component. The utilization of normalization is know to contribute decreased sensibility of learning process when ML models are trained. Overall, the resulting representation combines information about (i) the current construction progress, (ii) the remaining gap-constraint landscape, and (iii) global instance properties, yielding between 9 and 11 features depending on the configuration. This compact yet expressive encoding enables effective learning-based guidance of the search for high-quality variable-gap longest common subsequences.

3 Genetic Algorithm

The optimization process follows a population-based evolutionary scheme.

3.1 Initialization

Each individual is initialized by sampling weights uniformly from a predefined interval $[-w, w]$ (the value w parametrized, but fixed to 1).

3.2 Elitism

A fixed number of top-performing individuals are copied unchanged into the next generation.

3.3 Mutation

Several individuals are generated completely at random (random immigrants). This mechanism promotes exploration and maintains population diversity.

3.4 Offspring Generation

Three alternative parent-selection strategies are supported:

- **RKGA**: Two parents are selected uniformly at random, and each gene is inherited from either parent with probability 0.5.
- **BRKGA**: One parent is selected from the elite set and one from the non-elite set. Each gene is inherited from the elite parent with a predefined probability.
- **Lexicase Selection**: Training instances are randomly shuffled. Individuals are filtered iteratively by retaining those achieving the best performance on each instance. Parents are selected randomly from the remaining candidates.

We opt to use the *BRKGA* as the first option to work well.

3.5 Replacement and Termination

The new population consists of elites, mutants, and offspring. The algorithm terminates once a predefined time limit is reached (in this case we used 2 hours). The best individual encountered during training is continuously tracked and stored. (Q3) Terminate immediately when the validation value is decreased?

4 Overall Learning Procedure

The learning process alternates between evolutionary optimization of network weights and evaluation through (iterative multi-source) Beam Search. Importantly, the neural network does not directly predict solutions; instead, it learns how to *guide* the search procedure. This places the method within the class of neuro-evolutionary hyper-heuristics.

5 Algorithm

Algorithm 1 GA-Based Learning of a Beam Search Heuristic

```
Initialize population P with random neural network weights
Evaluate all individuals on training set T
Store best solution θ*
start ← current time
while time limit T_max has not exceeded do
    Sort P by fitness (descending) // Section 2.1
    Initialize empty population P'
    Copy top n_elites individuals from P to P'
    for i = 1 to n_mutants do
        Create random individual x
        Evaluate x on T
        Add x to P'
        Update θ* if improved
    end for
    for i = 1 to n_offspring do
        Select parents using RKGA, BRKGA, or Lexicase
        Generate offspring by crossover
        Evaluate offspring on T
        Add offspring to P'
        Update θ* if improved
    end for
    P ← P'
end while
return best weights θ*
```

As the best optimizer, the BRKGA setting used in training: $n_{pop} = 20, n_{elites} = 1, p_e = 0.5, p_m = 7$.

6 Discussion

The proposed framework learns heuristics rather than explicit solutions. The neural network is optimized exclusively through downstream search performance, enabling learning in non-differentiable settings. This tightly coupled integration of evolutionary learning and the classical optimization algorithm Beam Search allows the system to adapt its guidance strategy to the structure of the problem instances.

7 Preliminary results

IMSBS setting (grid search used for tuning the algorithm):

- $\beta = 1000$, $h' = h5$, $\#imbs_iters = 1000, \#num_roots = 10$

Learning IMSBS setting uses the same setting, to be completely fair with the comparison. The neural network configuration in this case, obtained after running several 2-layer (n_1, n_2) and 3-layer configurations (n_1, n_2, n_3) for $n_1, n_2, n_3 \in \{5, 10\}$ (neurons) and $n_1 \geq n_2 \geq n_3$. We set up the same activation function in all layers choosing among the three (most) known

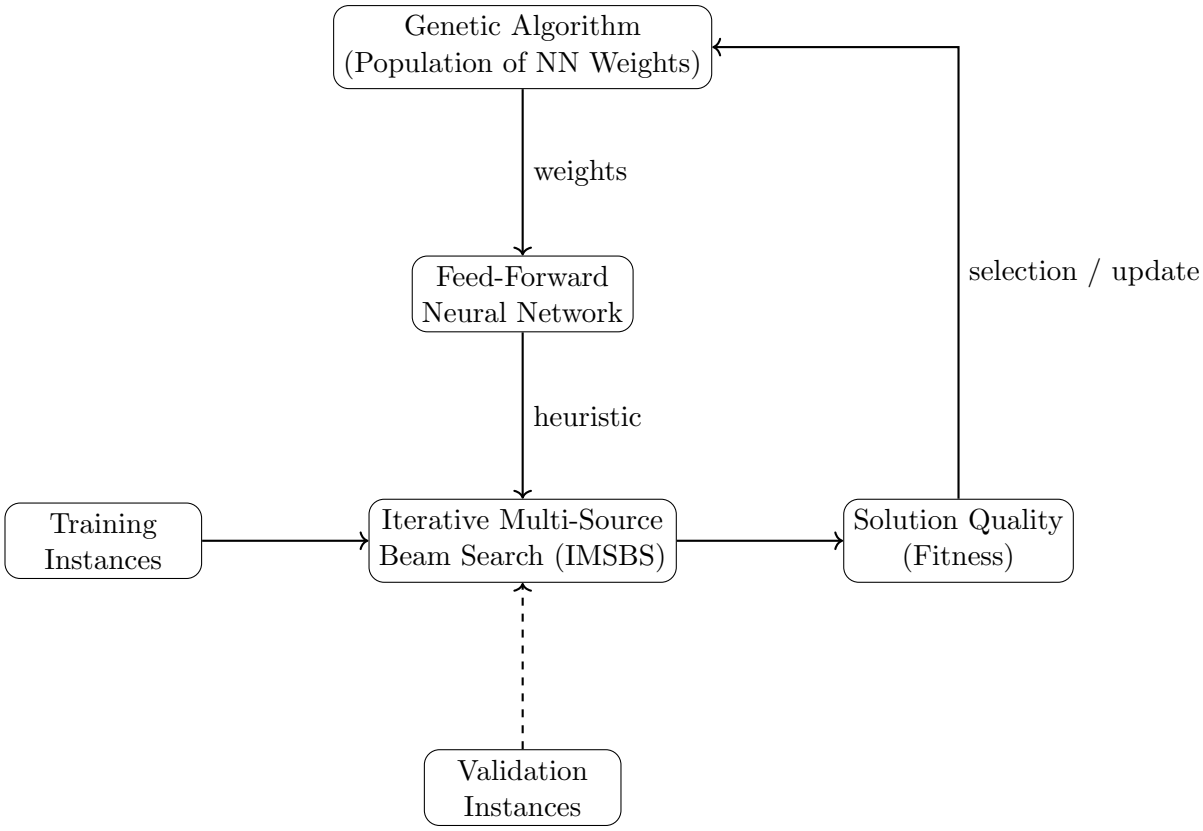

Figure 1: Learning framework integrating a Genetic Algorithm with Iterative Multi-Source Beam Search (IMSBS). The GA evolves neural network weights. For each individual, the FFNN provides a heuristic used by IMSBS on training instances. The resulting solution quality defines fitness, which drives evolutionary selection. Validation instances are used only for monitoring generalization.

$\{\texttt{tanh}, \texttt{ReLU}, \texttt{sigmoid}\}$. The best one uses `ReLU` at each layer, and besides instance-based features, it additionally incorporate the size of alphabet Σ (F2). The configurations of NNs are obtained only after 2 hours of training is a 3-layer perception with 10 neurons each.

8 Further ideas

(TODO: To be exploited)

Using only the mean optimizes average performance, ignores failures, and can produce unstable heuristics. In COPs, robustness is often more important than average performance.

Fitness function replacement:

- Option A (robust & simple) Fitness = $0.7 \cdot$ mean + $0.3 \cdot$ 90th-percentile
- Option B (very strong in COPs) Fitness = mean regret + 0.2 * std_dev
- Option C (robust learning): Fitness = 90th percentile

Curriculum / weighted fitness. Give more importance to harder instances and recent failures. Fitness=$\sum_i w_i f_i$, e.g., $w_i = \frac{1}{2^i}$, or $w_i = \frac{1}{n+1}$.

Multi-objective fitness. Instead of a single scalar: Objective 1: average quality, Objective 2: worst-case, runtime

Table 1: Comparison of objective values between IMSBS and LIMSBS. Best results per instance category are in bold.

m	n	$\lvert\Sigma\rvert$	IMSBS	LIMSBS
2	50	2	**37.7**	**37.7**
2	50	4	**30.1**	29.7
2	100	2	**72.9**	72.5
2	100	4	**61.7**	60.3
2	200	2	**139.7**	139.3
2	200	4	**125.2**	120.9
2	500	2	281.9	**295.4**
2	500	4	**302.9**	285.4
3	50	2	**31.2**	**31.2**
3	50	4	**22.9**	**22.9**
3	100	2	**58.5**	**58.5**
3	100	4	**48.5**	48.0
3	200	2	**106.3**	103.4
3	200	4	**97.4**	95.6
3	500	2	133.5	**143.1**
3	500	4	**230.3**	218.8

m	n	$\lvert\Sigma\rvert$	IMSBS	LIMSBS
5	50	2	**20.5**	**20.5**
5	50	4	**15.3**	**15.3**
5	100	2	**31.5**	31.0
5	100	4	25.9	**26.9**
5	200	2	35.2	**41.6**
5	200	4	34.0	**37.2**
5	500	2	31.0	**39.3**
5	500	4	36.3	**46.2**
10	50	2	**11.4**	11.2
10	50	4	**7.1**	**7.1**
10	100	2	10.5	**12.0**
10	100	4	7.6	**8.9**
10	200	2	11.8	**12.2**
10	200	4	7.6	**8.2**
10	500	2	12.5	**12.9**
10	500	4	7.5	**8.3**

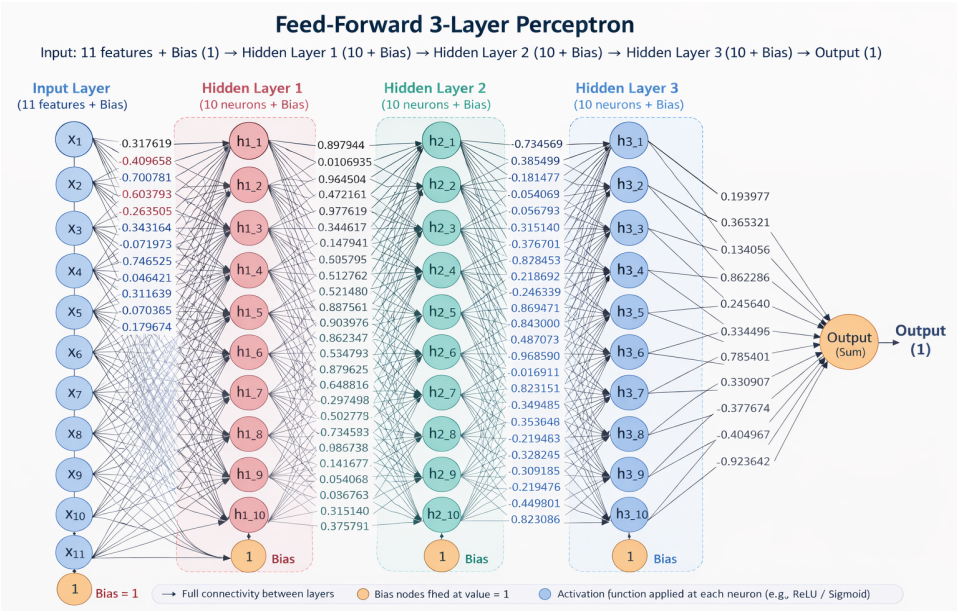

Figure 2: FFNN architecture that is learned in the neuro-evolution process.

Then use: Pareto ranking (common in evolutionary algorithms), particularly natural for Biased Random-Key Genetic Algorithms (BRKGA).